

Eco-design Digitale di Base per i servizi ICT

Programmazione in C e Python

Obiettivi della lezione

- Introduzione al Web e ai Server
- Introduzione ai Web Server con Python
- Flask: introduzione e setup
- Mini API rest
- Applicazioni e reali e deploy

Il Web e Internet

Internet

- **Rete globale di computer** interconnessi
- Infrastruttura fisica e logica
- Protocolli: TCP/IP, DNS, SMTP, FTP
- Nato negli anni '60-70 (ARPANET)

World Wide Web

- **Servizio** che gira su Internet
- Sistema di documenti ipertestuali
- Protocollo HTTP/HTTPS
- Inventato da Tim Berners-Lee nel 1989

Modello Client - Server

Client (Cliente)

- **Chi richiede** un servizio
- Browser web, app mobile, software
- Invia richieste e riceve risposte
- Esempi: Firefox, Chrome, Safari

Server (Servitore)

- **Chi fornisce** il servizio
- Computer sempre acceso e connesso
- Elabora richieste e invia risposte
- Esempi: Apache, Nginx, IIS

Modello Client - Server

Caratteristiche chiave:

- Comunicazione asincrona
- Il server può gestire più client contemporaneamente
- Architettura distribuita



Anatomia di un URL

`https://www.example.com:443/products/shoes?color=red&size=42#reviews`

Componenti:

- **Protocollo:** *https://* (come comunicare)
- **Dominio:** *www.example.com* (dove andare)
- **Porta:** *:443* (quale servizio, opzionale)
- **Percorso:** */products/shoes* (cosa cercare)
- **Query string:** *?color=red&size=42* (parametri)
- **Fragment:** *#reviews* (sezione specifica)
- **URI vs URL:** URI è il concetto generale, URL è un tipo specifico di URI che indica la posizione.

Protocollo HTTP

HTTP – HyperText Transfer Protocol

- È il **protocollo di comunicazione** usato per trasferire dati tra client (es. browser) e server web su Internet.
- 2 entità principali:
 - **REQUEST**
 - **RESPONSE**
- Metodi principali:
 - **GET, POST, PUT, DELETE**

Metodi HTTP

HTTP – HyperText Transfer Protocol

Metodi Principali

- **GET:** Recupera dati (sicuro, idempotente)
- **POST:** Crea nuove risorse (non idempotente)
- **PUT:** Aggiorna/sostituisce risorse (idempotente)
- **DELETE:** Elimina risorse (idempotente)

Altri Metodi

- **HEAD:** Come GET ma solo headers
- **PATCH:** Aggiornamento parziale
- **OPTIONS:** Scopre metodi supportati

Richiesta HTTP

Componenti:

- **Request Line:** metodo, percorso, versione HTTP
- **Headers:** metadati della richiesta
- **Body:** dati aggiuntivi (opzionale per GET)
- **Headers comuni:** Host, User-Agent, Accept, Content-Type, Authorizatio

```
GET /api/users/123 HTTP/1.1
```

```
Host: www.example.com
```

```
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
```

```
Accept: application/json
```

```
Authorization: Bearer token123
```

```
Connection: keep-alive
```

Risposta HTTP

Componenti:

- **Status Line:** versione HTTP, codice stato, messaggio
- **Headers:** metadati della risposta
- **Body:** contenuto effettivo

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 145
Set-Cookie: sessionId=abc123
Cache-Control: max-age=3600

{
  "id": 123,
  "name": "Mario Rossi",
  "email": "mario@example.com"
}
```

Ciclo di una richiesta HTTP

Flusso della Richiesta

1. **Client** digita URL o clicca link
 2. **DNS** risolve il nome in indirizzo IP
 3. **Connessione TCP** stabilita con il server
 4. **Richiesta HTTP** inviata al server
 5. **Server** elabora la richiesta
 6. **Risposta HTTP** inviata al client
 7. **Client** riceve e visualizza il contenuto
- **Tempo tipico:** 50-200ms per una richiesta locale, 200-1000ms per richieste intercontinentali.

Status Code HTTP

Categorie di Status Code

- **1xx:** Informational (100 Continue)
- **2xx:** Success (200 OK, 201 Created)
- **3xx:** Redirection (301 Moved, 304 Not Modified)
- **4xx:** Client Error (400 Bad Request, 404 Not Found, 401 Unauthorized)
- **5xx:** Server Error (500 Internal Error, 503 Service Unavailable)

I Più Comuni

- **200 OK:** Tutto perfetto
- **404 Not Found:** Risorsa non trovata
- **500 Internal Server Error:** Errore del server
- **403 Forbidden:** Accesso negato

DNS - Il Sistema dei Nomi

- **Domain Name System:** "rubrica di Internet"
- Traduce nomi in indirizzi IP
- Sistema gerarchico e distribuito
- Cache per migliorare le performance

Processo di Risoluzione

1. Browser controlla cache locale
 2. Query al DNS resolver dell'ISP
 3. Root servers → TLD servers → Authoritative servers
 4. Risposta con indirizzo IP
 5. Connessione diretta al server
- **Esempio:** `www.google.com` → `142.250.191.14`

Porte e indirizzi IP

- **Indirizzi IP**
- **IPv4:** 192.168.1.1 (32 bit)
- **IPv6:** 2001:db8::1 (128 bit)
- Identificano univocamente un dispositivo
- **Porte**
- **Numeri da 0 a 65535**
- Identificano servizi specifici su un server
- **Porte standard:** 80 (HTTP), 443 (HTTPS), 21 (FTP), 22 (SSH)
- **Combinazione**
- 192.168.1.1:8080 = server specifico + servizio specifico

FrontEnd vs BackEnd

Frontend

- **Interfaccia utente** che vede e usa
- Tecnologie: HTML, CSS, JavaScript
- Gira nel browser del client
- Responsabile di UX/UI

• Backend

- **Logica di business** e gestione dati
- Tecnologie: Python, Java, Node.js, PHP
- Gira sui server
- Gestisce database, autenticazione, API

FrontEnd vs BackEnd



L'Ecosistema Python Web

- Python offre diversi livelli di astrazione:
- **Livello Base:** http.server (standard library)
- Perfetto per prototipi e testing
- Controllo totale ma più codice da scrivere

Framework Leggeri: Flask

- Minimalista, flessibile
- Ideale per API e applicazioni piccole-medie

L'Ecosistema Python Web

- Python offre diversi livelli di astrazione:

Framework Completi: Django

- "Batteries included"

Framework Moderni: FastAPI

- Performance elevate
- Documentazione API automatica

WSGI vs ASGI

- **WSGI (Web Server Gateway Interface)**
- **Standard dal 2003**
- Gestione **sincrona** delle richieste
- Una richiesta alla volta per thread/processo
- Perfetto per applicazioni tradizionali

ASGI (Asynchronous Server Gateway Interface)

- **Standard moderno (2016+)**
- Gestione **asincrona** delle richieste → migliaia di connessioni simultanee
- Supporta WebSocket, HTTP/2, long-polling
- **Analogia:** WSGI è come una cassa del supermercato tradizionale, ASGI è come il self-checkout multiplo!

Primo server web

- Costruiamo insieme un server web da zero usando Python puro:

Cosa vedremo:

- Server HTTP basilare con `http.server`
- Gestione di GET e POST
- Servire file statici
- Routing semplice
- Rispondere con JSON

Framework Flask

Flask segue il principio "micro-framework":

- **Vantaggi:**
- Curva di apprendimento dolce
- Flessibilità massima
- Ecosistema ricco di estensioni
- Perfetto per API REST

Quando usarlo:

- API semplici e medie
- Prototipazione rapida
- Quando vuoi controllo totale sull'architettura

Django - Il Framework Completo

- Django è il "coltellino svizzero" del web Python:
- **Vantaggi:**
- Sistema di autenticazione robusto
- Admin panel automatico
- Sicurezza built-in (CSRF, XSS protection)
- Scalabilità enterprise

Quando usarlo:

- Applicazioni web complete
- E-commerce, CMS, social networks
- Quando hai bisogno di tutto subito
- Team di sviluppo grandi

Fast API

FastAPI combina performance e developer experience:

Vantaggi:

- **Performance:** tra le più alte in Python
- **Documentazione:** Swagger UI generata automaticamente
- **Asincrono:** supporto nativo async/await
- **Standards:** basato su OpenAPI e JSON Schema

Quando usarlo:

- API moderne ad alte performance
- Microservizi
- Applicazioni real-time
- Quando la velocità è critica

Quando Usare Cosa?

Flask quando:

- API semplici e veloci da sviluppare
- Hai bisogno di massima flessibilità
- Team piccolo, prototipazione
- Vuoi imparare i concetti web step-by-step



FastAPI quando:

Performance sono critiche

Stai costruendo API moderne

Vuoi documentazione automatica



Django quando:

- Applicazione web completa (frontend + backend)
- Hai bisogno di funzionalità enterprise (admin, auth, ORM)
- Team grande, progetti long-term

